UNIT-5

Importance of data Preparation?

Data preparation is essential in data science for handling large datasets because it transforms raw, messy data into a clean, accurate, and consistent format, ensuring reliable insights and accurate models, which in turn leads to better business decisions . This process involves correcting errors, handling missing values, integrating disparate sources, removing irrelevant data, and enriching the dataset, ultimately saving time and resources for the data scientist by streamlining analysis and improving computational efficiency.

1. Why Data Preparation is Crucial for Large Datasets

- **Improves Data Quality and Reliability**:

Raw data, especially from multiple large sources, often contains inconsistencies, missing values, errors, and irrelevant information. Data preparation addresses these issues, creating a trustworthy dataset for analysis.

- ◆ **Enhances Model Accuracy**:

Machine learning models are only as good as the data they are trained on. By providing clean and well-structured data, data preparation ensures models can accurately learn patterns, leading to better predictions and insights.

- **Increases Efficiency**:

While data preparation takes time upfront, it saves significant time and resources in the long run. It prevents time-consuming rework and troubleshooting later in the data pipeline, allowing data scientists to focus on analysis.

- **Facilitates Data Integration**:

In large-scale environments, data often comes from various sources in different formats. Data preparation standardizes these formats and resolves discrepancies, enabling a unified and comprehensive view of information.

- **Boosts Computational Efficiency**:

By reducing dataset size and eliminating redundant features, data preparation improves computational efficiency. This speeds up the analysis and modeling process, making it faster to iterate, experiment with algorithms, and optimize models.

- **Enables Better Decision-Making**:

Ultimately, reliable, accurate data leads to more informed business decisions. Data preparation ensures that the data used for analytics is robust enough to provide meaningful and actionable insights.

Key Aspects of Data Preparation for Large Data

- **Data Cleaning**:

Identifying and correcting errors, inconsistencies, and missing values.

3. **Data Transformation**:

Standardizing data formats, normalizing values, and applying transformations to make data suitable for analysis.

- **Feature Engineering**:

Creating new, more informative features from existing data to improve model performance.

- **Data Reduction**:

Compiled: Mr. Manikandan R
Compiled for EEE Final Year by Manikandan R

Reducing the dimensionality of data by removing irrelevant features to improve efficiency and reduce noise.

- **Data Integration**:

Consolidating data from multiple sources into a single, consistent dataset.

2. Give Detailed explanation and difference between Data parallelism and task parallelism

Data Parallelism

Data Parallelism means concurrent execution of the same task on each multiple computing core.

Data parallelism involves performing the same operation on different subsets of a large dataset concurrently. The core idea is to divide the data into smaller, independent chunks and assign each chunk to a different processor or core. Each processor then executes the identical operation on its assigned data portion simultaneously. This approach is highly effective when dealing with large volumes of data that require the same computation, such as image processing, scientific simulations, or machine learning training on massive datasets.

Let's take an example, summing the contents of an array of size N. For a single-core system, one thread would simply sum the elements [0] . . . [N − 1]. For a dual-core system, however, thread A, running on core 0, could sum the elements [0] . . . [N/2 − 1] and while thread B, running on core 1, could sum the elements [N/2] . . . [N − 1]. So the Two threads would be running in parallel on separate computing cores.

Task Parallelism

Task Parallelism means concurrent execution of the different task on multiple computing cores.

Task parallelism, also known as function parallelism or control parallelism, focuses on dividing a larger computational task into smaller, independent sub-tasks that can be executed concurrently. In this model, different processors or cores are assigned different sub-tasks, which may or may not operate on the same data. The sub-tasks can involve entirely distinct computations and may have dependencies, requiring coordination between processors. Task parallelism is beneficial for complex applications where different parts of the overall computation can be performed in parallel, such as in pipelined architectures or applications with multiple independent modules.

Consider again our example above, an example of task parallelism might involve two threads, each performing a unique statistical operation on the array of elements. Again The threads are operating in parallel on separate computing cores, but each is performing a unique operation.

The key differences between Data Parallelisms and Task Parallelisms are −

Data Parallelisms	Task Parallelisms
1. Same task are performed on different subsets of same data.	1. Different task are performed on the same or different data.
2. Synchronous computation is performed.	2. Asynchronous computation is performed.
3. As there is only one execution thread operating on all sets of data, so the speedup is more.	3. As each processor will execute a different thread or process on the same or different set of data, so speedup is less.
4. Amount of parallelization is	4. Amount of parallelization is proportional to the

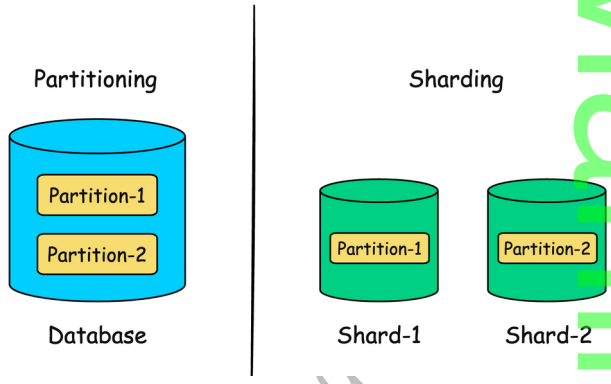
Compiled for EEE Final Year by Manikandan P
Compiled by Manikandan P

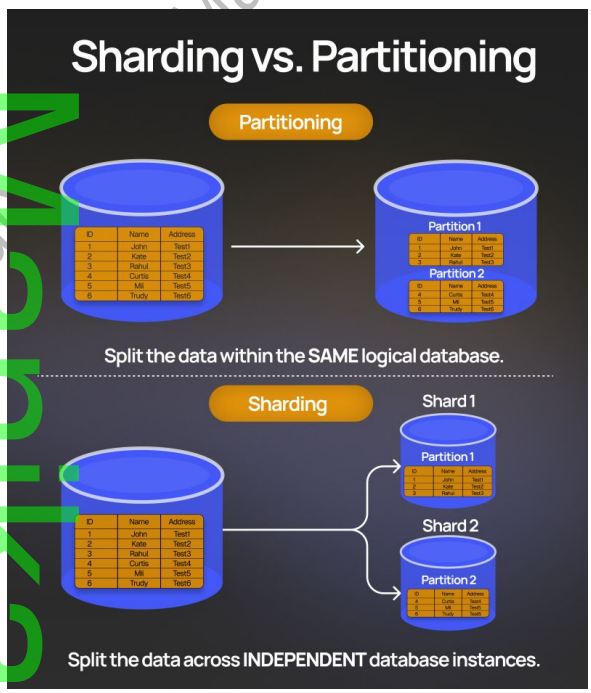
Data Parallelisms	Task Parallelisms
proportional to the input size.	number of independent tasks is performed.
5. It is designed for optimum load balance on multiprocessor system.	5. Here, load balancing depends upon on the e availability of the hardware and scheduling algorithms like static and dynamic scheduling.

3. Data partitioning and sharding

Database scalability is a system's capacity to handle growing workloads and data volumes by increasing resources without compromising performance or availability. This growth can be achieved by vertical scaling (adding more power, like CPU or RAM, to a single server) or **horizontal scaling** (adding more servers to distribute the workload). The goal is for the database to adapt to changing demands, expanding or contracting its resources to match user needs efficiently.

Scaling a server cluster is easy and flexible; you keep adding machines as the size of your data increases. Scaling a single machine, though, is comparatively difficult. **Sharding,** therefore, is more scalable than partitioning.





Sharding is the process of dividing a database into smaller, more manageable pieces called "shards." Each shard contains a subset of the overall data and functions as an independent database.

The shards are distributed across multiple servers, enabling the system to handle large datasets and high volumes of traffic. This approach balances the load among servers and allows for tailored optimizations for specific shards based on their data.

The following diagram illustrates how sharding works in a distributed database system. Notice how a load balancer and a database management system (DBMS) work together to distribute incoming client requests across multiple shards.

Compiled by Manikandan R - Manikandan R

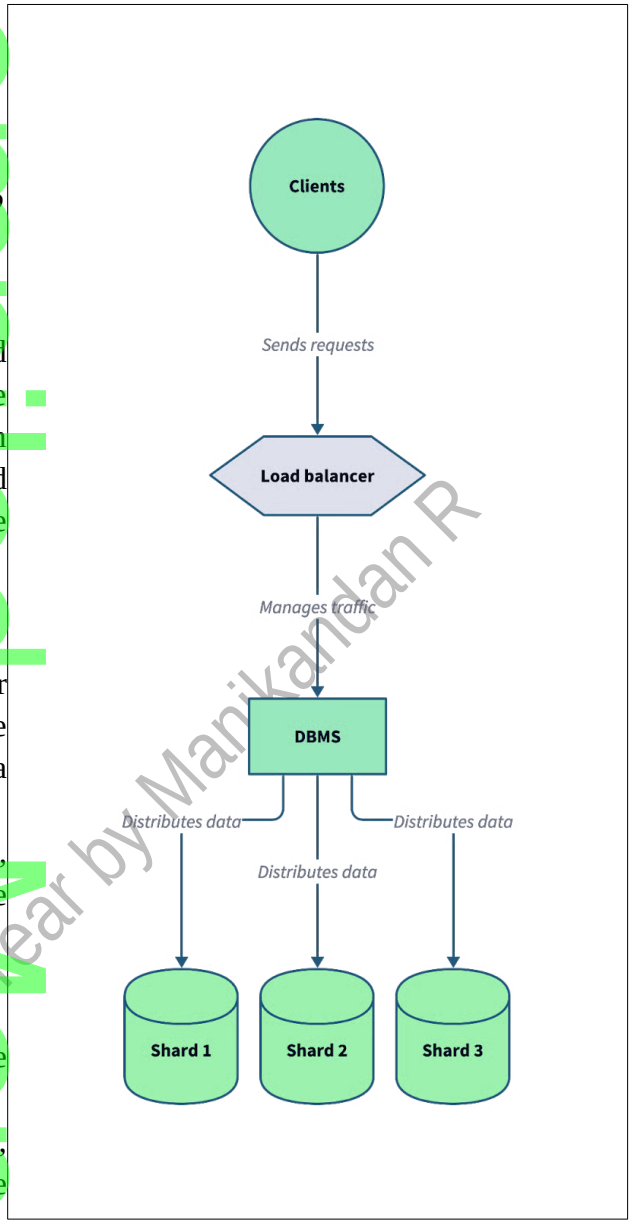
TOOLS

MongoDB - Built-in automatic sharding

Cassandra-Built-in sharding via consistent hashing
Amazon Redshift - Uses distribution keys to distribute data across nodes

Differences Between Sharding and Partitioning

Understanding the differences between sharding and partitioning is crucial for selecting the appropriate strategy to manage large datasets. While both techniques aim to optimize database performance and scalability, they operate at different levels and serve distinct purposes, as outlined below.



Scope and complexity

Sharding: Operates across multiple databases or servers, making it suitable for large-scale distributed systems. It can impact data on a more global scale.

Partitioning: Occurs within a single database, focusing on making a singular database more efficient as opposed to an entire cluster.

Data distribution

- **Sharding:** Distributes data across multiple nodes, enabling system-wide scalability.
- **Partitioning:** Does not distribute data by itself, but instead focuses on how that data should be divided.

Scalability

- **Sharding:** Supports horizontal scaling, handling increasing data volumes and user loads.
- **Partitioning:** Improves query performance but does not inherently scale across servers.

Management overhead

- **Sharding:** Requires complex management, including maintaining data consistency and handling distributed transactions.
- **Partitioning:** Easier to manage within a single database environment.

Use cases

- **Sharding:** Ideal for distributed, high-traffic applications like social media platforms and e-commerce systems.
- **Partitioning:** Best for scenarios requiring query optimization or efficient data archiving.

Compiled for EEE Final Year by Manikandan R
Compiled by Manikandan R

Sharding vs partitioning: A side-by-side comparison

Category	Sharding	Partitioning
Scope	Operates across multiple databases or servers	Happens within a single database
Complexity	Higher complexity: involves distributed architecture and coordination	Lower complexity: managed within one database system
Data distribution	Data is split and stored across different nodes/shards	Data is split into logical partitions within the same system
Scalability	Supports horizontal scaling by adding servers	Optimizes performance but doesn't inherently scale across servers
Management	Requires careful planning, custom tooling, and data consistency handling	Easier to maintain with built-in database features
Query performance	Depends on correct sharding key and data access patterns	Queries can be optimized automatically through partition pruning
Use cases	Best for large-scale, distributed apps (e.g., e-commerce, social media)	Ideal for analytical workloads and time-based/logical data queries

PARTITIONING

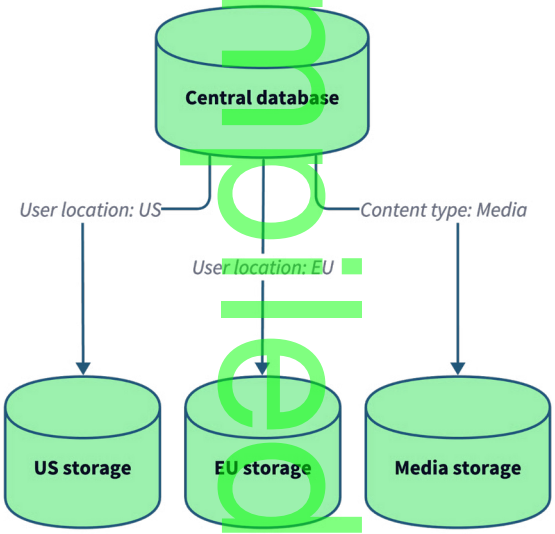
Partitioning is the process of dividing a large database table into smaller, more manageable segments called partitions—all within the same server and database system. Each partition holds a subset of the data based on a specified rule, such as date ranges, geographic regions, or customer IDs.

Unlike sharding, partitioning doesn't spread data across multiple machines. Instead, it helps organize data internally to speed up queries and simplify maintenance.But partitioning isn't just about organization—it directly impacts performance and data manageability. Here are some of its key benefits:

- **Query optimization:** Speeds up queries by limiting the search scope to a specific partition.
- **Efficient data management:** Simplifies data lifecycle management by segregating data for archiving or deletion.
- **Better indexing and maintenance:** Indexes can be applied at the partition level, reducing their size and making them easier to maintain. This keeps your database lean and responsive.

To better understand partitioning in action, let's look at a visual representation. In this example, data is stored in one central database but segmented into logical partitions based on user location or content type:

Compiled for EEE Final Year by Monikandan P
Compiled - Monikandan R



TOOLS WHICH SUPPORTS Partitioning

Amazon Redshift - Native support for columnar partitioning via sort and distribution keys

MySql - ***Native range, list, hash partitionin***

both mysql and amazon redshit supports both sharding and partitioning

4. Distributed computing frameworks for managing large data sets

*Key distributed computing frameworks for Big Data include **Apache Hadoop** for distributed storage and batch processing, **Apache Spark** for faster batch and real-time processing using in-memory computing, and **Apache Flink** for powerful stream and batch processing. Other important frameworks are **Apache Kafka** for real-time data streaming, **Apache HBase** for scalable NoSQL data, and **Dask** for distributed Python workloads*

Apache Spark:

- **Core Function:** A high-speed, general-purpose cluster-computing system.
- **Key Feature:** Offers in-memory processing, which is significantly faster than disk-based processing like MapReduce.
- **Best For:** Fast and versatile batch and real-time data processing, and advanced analytics.

Apache Flink:

- **Core Function:** An open-source framework for processing large streams of data.
- **Key Features:** Excels at stateful computations and can handle both real-time streaming and batch processing.

Compiled for EEE Final Year by Manikandan R - Manikandan R

- **Best For:** Low-latency, stateful applications and sophisticated real-time data processing.

Apache Kafka:

- **Core Function:** A distributed streaming platform for building real-time data pipelines and streaming applications.
- **Key Features:** High-throughput, fault-tolerant, and scales horizontally across many servers.
- ***Best For: Real-time data streaming and handling high-volume, continuous data feeds.***

Apache HBase:

- **Core Function:** A distributed, NoSQL database modeled after Google's Bigtable.
- **Key Feature:** Provides scalable, real-time read and write access to large datasets.
- **Best For:** Applications requiring fast, random access to large datasets.

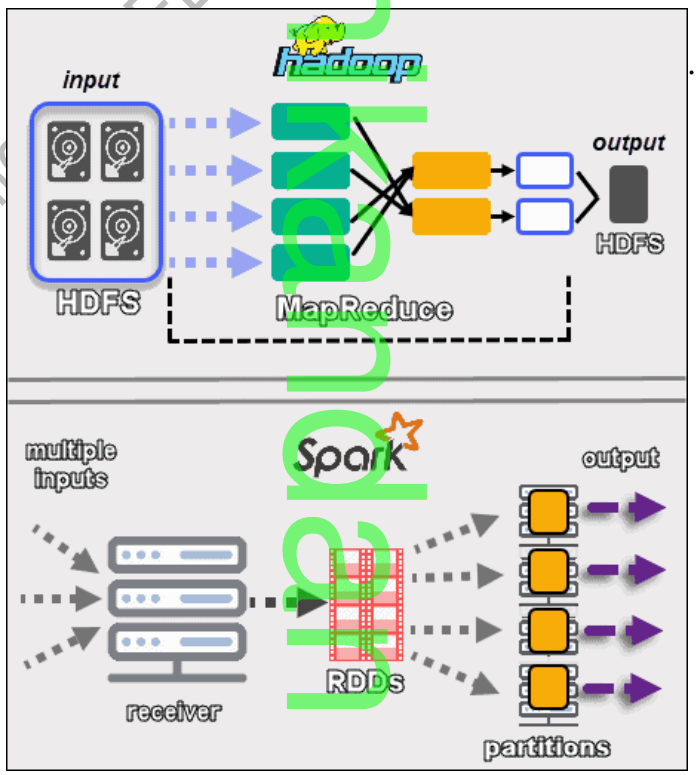
Dask:

- **Core Function:** A flexible library for parallel computing in Python.
- **Key Feature:** Scales Python workloads from a single machine to a cluster.
- **Best For:** Distributed data processing within the Python ecosystem.

Map Reduce

MapReduce is a big data analysis model that processes data sets using a parallel algorithm on computer clusters, typically Apache Hadoop clusters or cloud systems like Amazon Elastic MapReduce (EMR) clusters.

A software framework and programming model called MapReduce is used to process enormous volumes of data. Map and Reduce are the two stages of the MapReduce program's operation. Vast volumes of data are generated in today's data-driven market due to algorithms and applications



Compiled by EEE Final Year by Manikandan R
Compiled - Manikandan R

constantly gathering information about individuals, businesses, systems, and organizations. The tricky part is figuring out how to quickly and effectively digest this vast volume of data without losing insightful conclusions.

It used to be the case that the only way to access data stored in the Hadoop Distributed File System (HDFS) was using MapReduce. Other query-based methods are now utilized to obtain data from the HDFS using structured query language (SQL)-like commands, such as Hive and Pig. These, however, typically run alongside tasks created using the MapReduce approach.

Map Reduce is a framework in which we can write applications to run huge amount of data in parallel and in large cluster of commodity hardware in a reliable manner.

Phases of MapReduce

MapReduce model has three major and one optional phase.

- Mapping
- Shuffling and Sorting
- Reducing
- Combining

1) Mapping

It is the first phase of MapReduce programming. Mapping Phase accepts key-value pairs as input as (k, v), where the key represents the Key address of each record and the value represents the entire record content.The output of the Mapping phase will also be in the key-value format (k', v').

2) Shuffling and Sorting

The output of various mapping parts (k', v'), then goes into Shuffling and Sorting phase. All the same values are deleted, and different values are grouped together based on same keys. The output of the Shuffling and Sorting phase will be key-value pairs again as key and array of values (k, v[ ]).

3) Reducer

The output of the Shuffling and Sorting phase (k, v[]) will be the input of the Reducer phase. In this phase reducer function's logic is executed and all the values are Collected against their corresponding keys. Reducer stabilize outputs of various mappers and computes the final output

4) Combining It is an optional phase in the MapReduce phases. The combiner phase is used to optimize the performance of MapReduce phases. This phase makes the Shuffling and Sorting phase work even quicker by enabling additional performance features in MapReduce phases.

Compiled for EEE Final Year by Manikandan R
Compiled - Manikandan R

Numerical Example

We will be using **MovieLens Data.**

USER_ID	MOVIE_ID	RATING	TIMESTAMP
196	242	3	881250949
186	302	3	891717742
196	377	1	878887116
244	51	2	880606923
166	346	1	886397596
186	474	4	884182806
186	265	2	881171488

Solution

Step 1: First we have to map the values , it is happen in 1st phase of Map Reduce model.

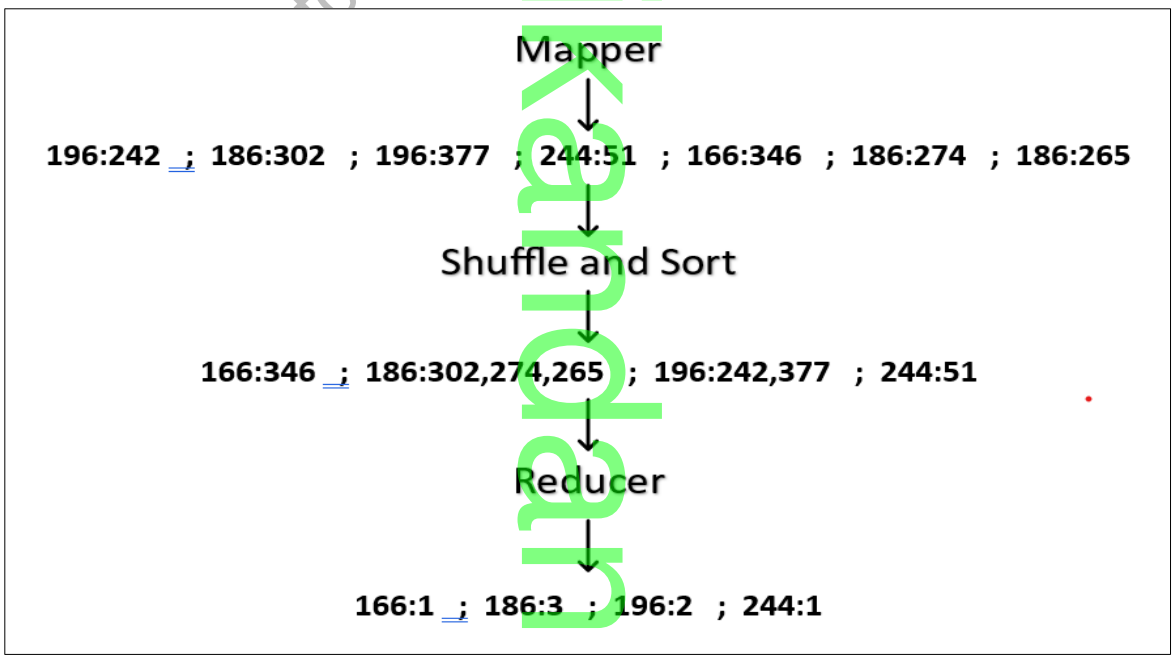
196:242 ; 186:302 ; 196:377 ; 244:51 ; 166:346 ; 186:274 ; 186:265

Step 2*:* After Mapping we have to shuffle and sort the values.

166:346 ; 186:302,274,265 ; 196:242,377 ; 244:51

Step 3*:* After completion of step1 and step2 we have to reduce each key's values.

Now, put all values together



Compiled - Manikandan R
for EEE Final Year by Manikandan R

Common Use Cases of MapReduce

- Counting word frequency (as shown above)
- Log analysis
- Indexing web pages
- Processing large datasets for ETL (Extract, Transform, Load)
- Recommendation systems and data mining

Advantages of MapReduce

- Simple and easy abstraction for large-scale data processing
- Efficient for batch processing of massive datasets
- Fault-tolerant and scalable
- Integrates well with Hadoop Distributed File System (HDFS)

Limitations of MapReduce

- Not ideal for real-time processing
- Complex data workflows can be hard to express
- Debugging and testing are more challenging
- High latency due to intermediate disk I/O (especially in the shuffle phase)

5. Describe about framework used to handle large dataset Apache Hadoop

Apache Hadoop is an open-source framework written in Java for distributed storage and processing of huge datasets. The keyword here is distributed since the data quantities in question are too large to be accommodated and analyzed by a single computer.

The framework provides a way to divide a huge data collection into smaller chunks and shove them across interconnected computers or nodes that make up a Hadoop cluster. As a result, a Big Data analytics task is split up, with each machine performing its own little part in parallel. Still, an end-user sees all the fragments as a single unit.

Hadoop hides away the complexities of distributed computing, offering an abstracted API to get direct access to the system's functionality and its benefits:

- **scalability.** You can quickly add new nodes to the cluster, scaling it up from a single computer for proof-of-concept to hundreds of machines. Hadoop puts virtually no limits on the storage capacity.
- **versatility.** Hadoop allows you to leverage data from multiple sources and in different formats, both structured and unstructured. You don't need to archive or clean data before loading.
- **cost-effectiveness.** Hadoop works on low-cost, commodity hardware, making it relatively cheap to maintain.
- **fail-safe design.** The system automatically replicates information to prevent data loss in the case of a node failure.

To understand how the entire mechanism works, we need to get familiar with Hadoop structure and key parts.

Hadoop architecture, or how the framework works

There are two ways to deploy Hadoop —

Compiled for EEE Finalyear by Manikandan R
Compiled - Manikandan R

as a *single-node cluster* - In the *single-node cluster*, the framework is set up on one virtual machine, which is preferable for the evaluation or test phase.

as a *multi-node cluster.* *-more common, scenario, each node runs on a separate virtual machine. Obviously, Big Data processing involves hundreds of computing units.* So, further reading refers to a multi-node deployment option.

Hadoop nodes: masters and slaves

Master Nodes control and coordinate two key functions of Hadoop: data storage and parallel processing of data. Physically, they require the best hardware resources available.

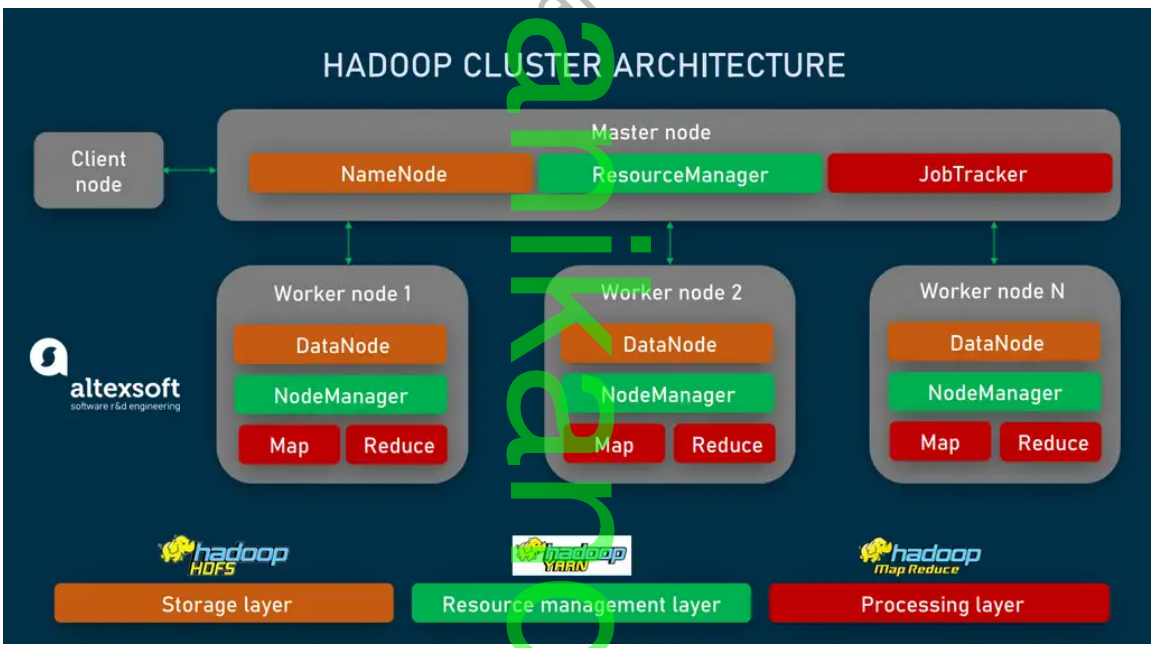
Worker or Slave Nodes are the majority of nodes used to store data and run computations according to instructions from a master node.

A Client Node also called **a Gateway or Edge Node** acts as an interface between a Hadoop cluster and an outside network. It doesn't belong to the master-slave paradigm, being responsible for loading data into the cluster, describing how the data must be processed, and retrieving the output.

Each **Hadoop cluster has three functional layers:**

- the storage layer created by Hadoop's native file system — **HDFS**,
- the resource management layer represented by **YARN**, and
- the processing layer called **MapReduce**.

All Hadoop layers are built around master/worker interactions



HDFS: a storage layer

The backbone of the framework, **Hadoop Distributed File System** (HDFS for short) stores and manages data that is split into blocks across numerous computers. By default, the block size in Hadoop is 128MB, but this can be easily changed in the config file.

Compiled by Manikandan R
Year by Manikandan R

HDFS works on the *"write once, read many times"* principle. A file stored in the system can't be modified but it can be analyzed for different purposes again and again. Following this approach, the tool focuses on fast retrieval of the whole data set rather than on the speed of the storing process or fetching a single record.

Each block of HDFS is automatically replicated in different worker nodes to ensure *fault tolerance*. If a node with required data fails, you can always make use of a backup. The recommended number of replicas is three, with "no more than one copy on the same node and no more than two copies in the same rack."

HDFS master-slave structure

An **HDFS Master Node**, called a **NameNode**, keeps metadata with critical information about system files (like their names, locations, number of data blocks in the file, etc.) and keeps track of storage capacity, the volume of data being transferred, etc.

Multiple Worker Nodes — DataNodes — house blocks of large files. Every three seconds workers send signals to their master to inform it that everything goes well and data is ready to be accessed.

DataNodes are organized in **racks** of 40-50 nodes connected to the same network switch.

YARN: a resource management layer

An acronym for **Yet Another Resource Negotiator, YARN is** a software layer that monitors the usage of CPU, memory, and disk space, allocates resources to running applications, and schedules jobs based on the application requirements.

YARN master-slave structure

A **ResourceManager** serves as a Master Node and has ultimate authority over resources in the system.

Multiple slaves — **NodeManagers** — monitor resources of each virtual machine, reporting results to the ResourceManager.

MapReduce: a processing layer

MapReduce is often recognized as the best solution for batch processing, when files gathered over a period of time are automatically handled as a single group or batch.

The entire job is divided into two phases: map and reduce (hence the name.) **Map operations deal with data filtering, sorting, and splitting** while the **Reduce stage takes care of aggregating and summarizing results.**

This approach fits companies that want to extract detailed insights from large data volumes — rather than to get fast analytics results in real-time.

MapReduce master-slave structure

A Master Node known as a **JobTracker** receives client requests via a client node, connects with the NameNode to find out the data location, assigns tasks to slave nodes and keeps the client informed about the job status.

Slave Nodes or **TaskTrackers** perform map and reduce tasks according to the JobTracker instructions. Similar to DataNodes, they are constantly informing their Master Node on the execution progress.

Compiled for EEE Final Year by Manikandan R
Compiled - Manikandan R

Hadoop limitations

A powerful Big Data tool, Apache Hadoop alone is far from being all-powerful. It has multiple limitations. Below we list the greatest drawbacks of Hadoop.

Small file problem. Hadoop was created to deal with huge datasets rather than with a large number of files extremely smaller than the default size of 128 MB. For every data unit, the NameNode has to store metadata with names, access rights, locations, and so on. Millions of small files will evidently occupy too much memory in the Master Nodes and create lots of tasks that will slow down the processing.

High latency of data access. Hadoop ensures high throughput which means the system's ability to deliver large data batches. But this comes at the expense of latency — or delay between user action and system response. In other words, it will take relatively long to find and retrieve a single record. High latency makes Hadoop unsuitable for tasks that require nearly real-time data access.

No real-time data processing. MapReduce performs batch processing only and doesn't fit time-sensitive data or real-time analytics jobs.

Complex programming environment. Data engineers who previously worked only with relational database management systems and SQL queries need training to take advantage of Hadoop. They have to know Java to go deep in Hadoop coding and effectively use features available via Java APIs. It's also important to understand the core principles behind Hadoop.

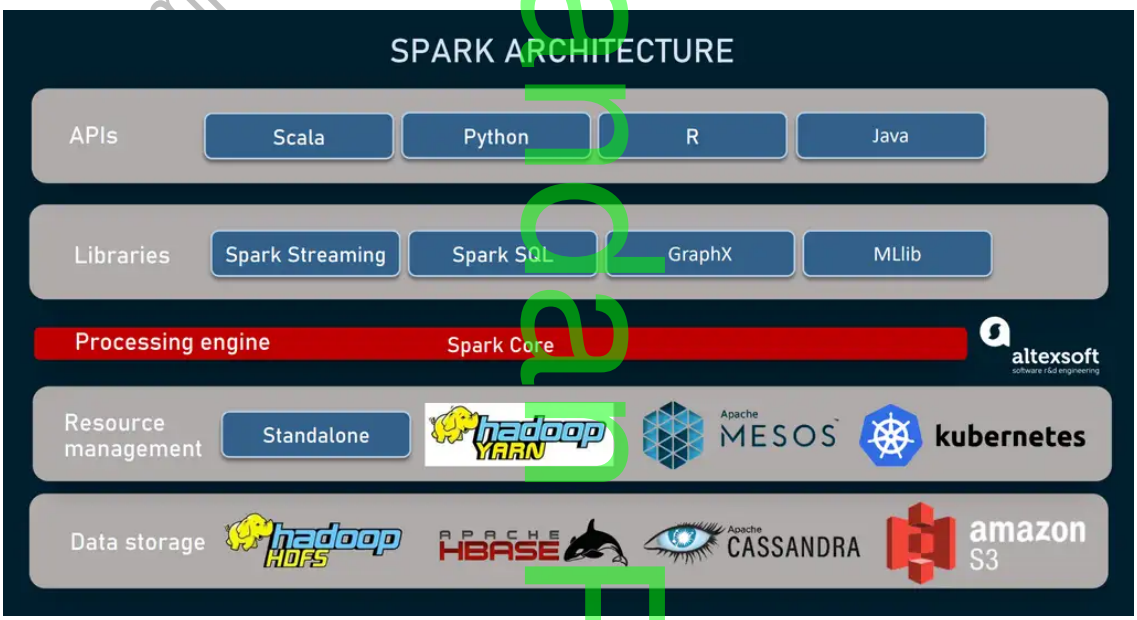
Apache Spark

Designed specifically to replace MapReduce, Spark also processes data in batches, with workloads distributed across a cluster of interconnected servers.

Similar to its predecessor, the engine supports single- and multi-node deployment scenarios and master-slave architecture. Each Spark cluster has a single master node or *driver* to manage tasks and numerous slaves or *executors* to perform operations. And that's almost where the likeness ends.

The main difference between Hadoop and Spark lies in data processing methods.

MapReduce stores intermediate results on local discs and reads them later for further calculations. In contrast, Spark caches data in the main computer memory or RAM (Random Access Memory.)



Compiled for EEE Final Year by Manikandan R

Even the best possible disk read time lags far behind RAM speeds. Not a big surprise that Spark runs workloads 100 times faster than MapReduce if all data fits in RAM. When datasets are so large or queries are so complex that they have to be saved to disc, Spark still outperforms the Hadoop engine by ten times.

Spark Core and distributed data structures

The heart of the framework is its computational engine known as Spark Core. It is responsible for

- distributed data processing,
- memory management,
- task scheduling,
- fault recovery, and
- communications with an external cluster manager and data repository.

The **fundamental data structure Spark Core works with is *Resilient Distributed Dataset (RDD.)*** Basically, it's a read-only and fault-tolerant collection of records that can be processed in parallel, hiding partitioning from an end-user. RDD easily handles both structured and unstructured data.

Another available schema — *DataFrames* — is used to organize information in the named columns, similar to tables in relational databases.

Its extension called *Datasets* merges benefits of the two previous models. It supports all types of data like RDD and at the same time allows for performing SQL queries — though it happens more slowly than with DataFrames.

No default storage system and resource manager

Unlike Hadoop, which unites storing, processing, and resource management capabilities, Spark is for processing only, having no native storage system. Instead, it can read and write data from/to different sources, including but not limited to **HDFS, HBase, and Apache Cassandra.** It is compatible with a plethora of other data repositories, outside the Hadoop ecosystem — say, **Amazon S3**.

Processing data across multiple servers, Spark couldn't control resources — mainly, CPU and memory — by itself. For this task, it needs a resource or cluster manager. Currently, the framework supports four options:

- Standalone, a simple pre-built cluster manager;
- Hadoop YARN, which is the most common choice for Spark;
- Apache Mesos, used to control resources of entire data centers and heavy-duty services; and
- Kubernetes, a container orchestration platform.

Running Spark on Kubernetes makes sense if a company plans to move the entire company techstack to the cloud-native infrastructure.

Native libraries: Spark Streaming, Spark SQL, MLlib, and GraphX

Spark limitations

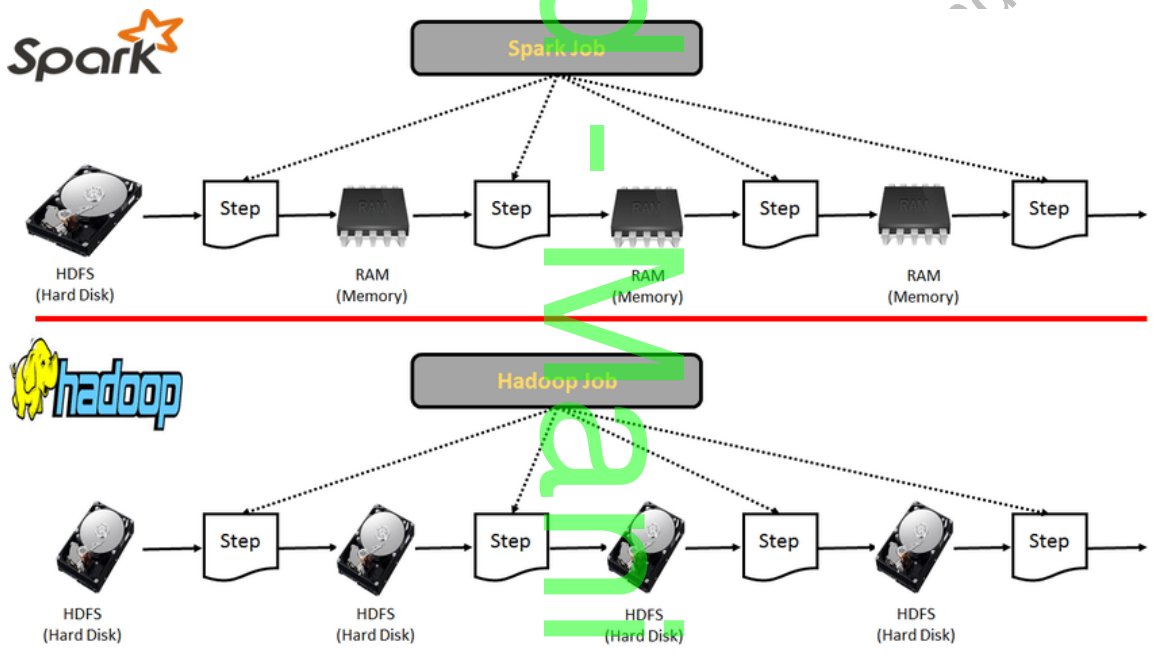
Obviously, Spark has some advantages over Hadoop's MapReduce engine. Yet, it also comes with certain drawbacks you should consider.

Compiled for EEE Final Year by Manikandan R
Compiled by Manikandan R

Pricey hardware. RAM prices are higher than those of hard discs exploited by MapReduce, making Spark operations more expensive.

Near, but not truly real-time processing. Spark Streaming and in-memory caching allow you to analyze data very quickly. But still it won't be truly real-time, since the module works with micro-batches — or small groups of events collected over a predefined interval. Genuine real-time processing tools process data streams at the moment they are generated.

Issues with small files. Like Hadoop, Spark doesn't cope well with a large number of small datasets. More files within a workload mean more metadata to parse and more tasks to schedule, which can slow up the processing dramatically.



Out of core machine learning

6. Several popular Python libraries are utilized for out-of-core machine learning, which involves processing datasets too large to fit into RAM. These libraries employ various techniques to handle large data efficiently.

Dask: Dask provides parallel computing capabilities for Python, allowing for out-of-core computations on arrays, DataFrames, and custom data structures. It integrates well with other libraries like NumPy and Pandas, enabling scalable machine learning workflows.

Vaex: Vaex is a high-performance Python library specifically designed for lazy, out-of-core DataFrames. It allows for computations on datasets with billions of rows, offering efficient memory usage and fast statistical calculations and visualizations.

Compiled by Manikandan R

PySpark: While not exclusively a Python library, PySpark provides a Python API for Apache Spark, a powerful distributed computing framework. Spark is highly effective for processing large datasets in a distributed manner, making it suitable for out-of-core machine learning, especially in big data environments.

Scikit-learn (with careful usage): Although Scikit-learn primarily focuses on in-memory algorithms, some of its functionalities can be adapted for out-of-core learning. For instance, `partial_fit` methods available in certain estimators (e.g., `SGDClassifier`, `PassiveAggressiveClassifier`) allow for incremental learning on mini-batches of data, making them suitable for datasets that cannot be loaded entirely into memory.

TensorFlow/PyTorch (for deep learning): In deep learning, frameworks like TensorFlow and PyTorch handle large datasets by loading data in mini-batches and utilizing data generators or `tf.data` pipelines to efficiently stream data from disk during training, effectively achieving out-of-core processing for neural networks.

7. Tools & Techniques: Handling Large Data in Data Science

Main Branches:

- **Data Ingestion & Storage:**
  - **Techniques:** Distributed file systems, streaming data ingestion, data warehousing, data lakes.
  - **Tools:**
    - **Distributed File Systems:** HDFS (Hadoop Distributed File System)
    - **Streaming Platforms:** Apache Kafka, Apache Flink
    - **Data Warehouses:** Snowflake, Amazon Redshift, Google BigQuery
    - **Data Lakes:** Amazon S3, Azure Data Lake Storage, Google Cloud Storage
    - **NoSQL Databases:** Apache Cassandra, MongoDB
- **Data Preprocessing & Cleaning:**
  - **Techniques:** Data sampling, dimensionality reduction, feature scaling, handling missing values, outlier detection, data transformation.
  - **Tools:**
    - **Programming Languages:** Python (Pandas, Dask), R
    - **Distributed Processing Frameworks:** Apache Spark, Apache Flink
    - **ETL Tools:** Apache NiFi, Talend
- **Exploratory Data Analysis (EDA) & Visualization:**
  - **Techniques:** Summary statistics, data visualization (scatter plots, histograms, heatmaps), correlation analysis, interactive dashboards.
  - **Tools:**
    - **Programming Libraries:** Matplotlib, Seaborn, Plotly, Bokeh (Python), ggplot2 (R)

Compiled for EEE Final Year by Manikandan R

    - **Business Intelligence Tools:** Tableau, Power BI, Looker
    - **Distributed Computing for EDA:** Dask, Spark with libraries like Spark SQL
- **Model Building & Training:**
  - **Techniques:** Distributed machine learning algorithms, parallel processing, model selection, hyperparameter tuning.
  - **Tools:**
    - **Distributed ML Libraries:** Apache Spark MLlib, TensorFlow Distributed, PyTorch Distributed
    - **Cloud ML Platforms:** Google AI Platform, AWS SageMaker, Azure Machine Learning
    - **Specialized Libraries:** XGBoost, LightGBM (can be used in distributed environments)
- **Model Evaluation & Deployment:**
  - **Techniques:** Performance metrics (accuracy, precision, recall), A/B testing, model monitoring, containerization, API deployment.
  - **Tools:**
    - **Containerization:** Docker, Kubernetes
    - **Model Serving:** TensorFlow Serving, TorchServe, BentoML
    - **Monitoring Tools:** Prometheus, Grafana
- **Orchestration & Workflow Management:**
  - **Techniques:** Automating data pipelines, scheduling tasks, managing dependencies.
  - **Tools:** Apache Airflow, Luigi, Prefect

This mind map provides a structured overview of the key areas and associated tools/techniques for handling large datasets throughout the data science lifecycle.

8. Malicious URL detection

This pipeline is designed to handle **millions of URLs per day**, requiring a highly scalable, fault-tolerant, and low-latency system.

1. Architecture Diagram

Here is a high-level overview of the end-to-end pipeline:

2. Component Justification & Tool Choices

Component	Tool Choice	Justification
Data Ingestion (Real-time Stream)	**Apache Kafka**	High-throughput, fault-tolerant, distributed log for ingesting millions of URL requests/logs. Decouples the producers (web servers) from the consumers (feature extractors/storage).
Data Storage (Raw/Historical)	**Apache Cassandra**	Horizontally scalable, distributed NoSQL database. Ideal for storing vast amounts of historical URL data (raw and processed features) for **offline training** and batch feature lookups.

Compiled for EEE Final Year by Manikandan R
Compiled by: Manikandan R

Component	Tool Choice	Justification
Batch Feature Extraction	**Apache Spark**	A unified engine for large-scale data processing. Used for **offline** ETL, featurization of the historical dataset, and model training on Terabytes of data.
Feature Store / Cache	**Redis**	In-memory data structure store used as a low-latency **online** feature store and serving cache. Essential for real-time feature lookups during prediction.
Model Training (Offline)	**Spark MLlib / TensorFlow/PyTorch**	Spark MLlib is used for large-scale model training. TensorFlow/PyTorch is used for more complex models (e.g., character-level CNNs/RNNs).
Model Deployment & Serving (Online)	**Kubernetes (K8s) & Model Server (e.g., TensorFlow Serving/TorchServe/Bento ML)**	K8s provides orchestration, auto-scaling, and health checks for the prediction microservice. A dedicated model server ensures low-latency, high-throughput model inference.
Monitoring	**Prometheus & Grafana**	Prometheus for metrics collection (latency, error rate, model performance metrics like AUC/F1). Grafana for visualization and alerting.

Malicious URL Detection Pipeline at Scale

The pipeline is split into an **Offline Training Loop** (batch processing) and an **Online Serving Loop** (real-time prediction).

Malicious URL Detection Pipeline at Scale

The pipeline is split into an **Offline Training Loop** (batch processing) and an **Online Serving Loop** (real-time prediction).

---

1. Tool Justification

Component	Tool Choice	Role in Pipeline	Justification
Data Ingestion	**Apache Kafka**	Real-time queue for raw URL requests and prediction logs.	High-throughput, fault-tolerant, and durable messaging system. Decouples the low-latency prediction service from the high-throughput upstream applications.
Historical Storage	**Apache Cassandra**	Stores raw and labeled historical URL data.	Horizontally scalable, distributed NoSQL database. Excellent for write-heavy data ingestion and time-series data storage, ideal for Terabytes of historical data needed for batch training.

Compiled for EEE Final Year by Manikandan R
Compiled by Manikandan R

Component	Tool Choice	Role in Pipeline	Justification
Batch Processing	**Apache Spark**	Executes ETL (Extract, Transform, Load) and model training tasks.	Unified engine for large-scale data processing. Handles the Terabyte-scale feature engineering and parallelized model training efficiently.
Feature Store (Online Cache)	**Redis**	Low-latency key-value store for static features.	In-memory data store for sub-10ms lookups of pre-calculated, slow-changing features (e.g., domain age), critical for low-latency online serving.
Model Serving	**Kubernetes (K8s) + TensorFlow Serving**	Orchestrates and serves the trained model as a microservice.	K8s provides automatic scaling and health checks. TensorFlow Serving offers high-performance, concurrent, and versioned model inference.

2. Feature Extraction at Scale (Offline Spark)

Feature engineering is crucial. Spark handles the large-scale transformation of raw URLs into numeric feature vectors.

Feature Type	Example	Scaling Technique
Lexical	URL length, num_dots entropy.	Calculated using Spark UDFs (User-Defined Functions).
Host-based	Domain_Age, TTL (Time-To-Live).	Lookups/joins with external WHOIS data sources.
Content/Text	Character N-grams, path_tokens.	**Hashing Trick/HashingTF:** Uses a fixed-size vector based on a hash function, avoiding a massive vocabulary dictionary.

Worked Example: Feature Vectorization (Text)

For the *Content/Text* features, we use the Hashing Trick for scalability, avoiding memory issues with a huge dictionary of all possible n-grams.

1. **Input URL:** `http://bad.site.ru/login.php`
2. **Character N-grams (n=3):** ‘htt‘, ‘ttp‘, ‘tp:‘, ‘p:/‘, ..., ‘n.p‘, ‘.ph‘, ‘php‘
3. **Hashing Trick:** Map each n-gram to an index i in a fixed-size vector V using a hash function h(ngram) and vector size N. i=h(ngram)(modN).

- **Pseudocode (Spark HashingTF Concept):**

```
# Set vector size N (e.g., 2^18 = 262144) to balance collision risk vs. memory.
VECTOR_SIZE = 262144

# 1. Tokenize URL into character n-grams (Spark UDF)
url_df = url_df.withColumn("ngrams", n_gram_tokenizer("raw_url", n=3))

# 2. Apply HashingTF to convert tokens to a fixed-size feature vector
```

Compiled for EEE Final Year by Manikandan R
Manikandan R

```
hasher=HashingTF(inputCol="ngrams", outputCol="features_text", numFeatures=VECTOR_SIZE)
featurized_df = hasher.transform(url_df)

# 3. Combine with other features (Lexical, Host) using VectorAssembler
final_features_df = VectorAssembler(inputCols=["features_lexical", "features_text"],
                                    outputCol="features").transform(featurized_df)
```

3. Model Serving (Online Loop)

The goal is high-concurrency, low-latency prediction.

1. **Request:** External application sends URL to the K8s → Prediction Service.
2. **Feature Retrieval:** The service calculates simple features on-the-fly. It fetches pre-calculated static features (e.g., Domain_Age) from **Redis** (typically < 5ms).
3. **Prediction:** The assembled feature vector is passed to the TensorFlow Serving instance running the trained GBM or CNN model.
4. **Prediction Log:** The raw URL, features, and the prediction score are sent to a Kafka log for monitoring and eventual retraining data collection.

Latency Derivation (Target: <100ms)

The total latency τ_{total} is the sum of components:

$$\tau_{total}=\tau_{network}+\tau_{redis_lookup}+\tau_{on_the_fly_feat}+\tau_{inference}$$

- $\tau_{network}\approx 20ms$
- $\tau_{redis_lookup}\approx 5ms$ (critical for speed)
- $\tau_{on_the_fly_feat}\approx 10ms$
- $\tau_{inference}\approx 20ms$ (using a fast model like GBM or Lightweight DNN)

$$\tau_{total}\approx 20+5+10+20=55ms$$

This is well within the target, allowing the service to handle high QPS (Queries Per Second).

4. Monitoring and Retraining Triggers

The system requires continuous monitoring due to the adversarial nature of malicious URLs.

Metric Type	Example Metric	Tool	Retraining Trigger
Model Performance	F1-score AUC on verified predictions.	Grafana/Prometheus	**Concept Drift:** F1-score on new data drops below 0.95.
Data Quality/Drift	Distribution of URL length, new TLDs.	Spark analysis jobs on Kafka logs.	**Data Drift:** A feature's distribution diverges significantly (e.g., K-S test p-value < 0.01) from the training baseline.

Compiled for EEE Final Year by Manikandan R
Manikandan R

Metric Type	Example Metric	Tool	Retraining Trigger
Operational	Prediction Latency, QPS, Error Rate.	Prometheus	**Operational Issue:** P99 latency exceeds 150ms for 10min.

The Retraining Loop automatically re-runs the Offline Training Pipeline upon a critical trigger, trains the new model, and deploys it to the K8s/TF Serving environment (A/B testing or rolling update) for continuous improvement.

Recommender System

NETFLIX RECOMMENDATION SYSTEM

Netflix uses an AI-powered recommendation system that learns from subscribers or users habits to serve spot-on suggestions. This AI-driven engine curates personalized suggestions for each user based on a rich mix of viewing behavior, content preferences, and contextual signals. Over 80% of all viewing activity on Netflix comes from these tailored suggestions.

In this case study, we will break down how this powerful system works, what its algorithms are, the tech details behind the scenes, and the impact this system has. let's get into the background of the popular OTT.

Table of Content

- Background of Netflix's Content Challenge
- How Netflix's Recommendation System Works?
- Data Collection
- Algorithms and Techniques
- Personalization Features on Netflix
- Technical Infrastructure and Architecture
- Evaluation and Testing
- Impact on User Experience and Business
- Challenges and Future Directions

Background of Netflix's Content Challenge

Netflix began in 1997 as a DVD rental service and, within a decade, evolved into a full-fledged streaming platform by 2007. As its content library rapidly grew, a new challenge surfaced: how to help users easily discover shows and movies they'd truly enjoy. With thousands of titles available and over 260 million subscribers worldwide, it quickly became clear that personalization was no longer optional; it was critical. A generic, one-size-fits-all interface risked overwhelming users and diluting the viewing experience. To solve this, Netflix invested heavily in building a personalized recommendation system - one that could intelligently adapt to each viewer's unique tastes, preferences, and watching habits.

Compiled for EEE Final Year by Manikandan R
Compiled by Manikandan R

How Netflix's Recommendation System Works?

Netflix's recommendation system starts with a single, powerful question: What does each viewer want, even before they know it? To answer that, it taps into two rich sources of insight: user behavior and content metadata. Let's break each of them down.

Data Collection

User Behavior: Netflix collects extensive data on user interactions, including:

- Viewing history: What a user watches, including the duration and frequency.
- Ratings: User ratings for watched content.
- Search queries: What users search for on the platform.
- Browsing behavior: Navigation patterns, click-through rates, and scroll depths.
- Interaction timestamps: When content is watched (time of day, day of the week).

Content Metadata: This includes detailed information about the content itself:

- Genre, sub-genre, and micro-genres.
- Cast, director, and crew information.
- Release year and content duration.
- User-generated tags and descriptions.
- Detailed synopsis and themes.

Algorithms and Techniques

Netflix uses a combination of classic and cutting-edge machine learning methods:

Collaborative Filtering

- **User-Based Collaborative Filtering**: Recommendations are made based on the preferences of similar users. If User A and User B have similar viewing habits, content enjoyed by User A is likely to be recommended to User B.
- **Item-Based Collaborative Filtering**: This method looks at the relationship between items. If two items are often watched together, they are likely to be recommended in tandem.
- **Matrix Factorization**: Techniques like Singular Value Decomposition (SVD) break down large matrices (user-item interactions) into lower-dimensional spaces to uncover latent factors that influence preferences.

Content-Based Filtering

- This method recommends content similar to what a user has watched in the past. For instance, if a user enjoys science fiction movies, the system suggests more titles in the same genre.
- **Natural Language Processing (NLP)**: Used to analyze and understand the content metadata and user reviews to extract meaningful features.

Deep Learning and Neural Networks

- **Convolutional Neural Networks (CNNs)**: Used for image processing, helping to analyze and recommend based on video frames and thumbnails.
- **Recurrent Neural Networks (RNNs) and Long Short-Term Memory Networks (LSTMs)**: Effective in capturing temporal patterns in viewing behavior, making predictions based on sequential data.
- **Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs)**: Used for generating new content recommendations by learning complex distributions of user preferences.

Compiled for EEE Final Year by Manikandan R
Compiled - Manikandan R

Bandit Algorithms

- Multi-armed bandit algorithms are employed for exploration-exploitation trade-offs, balancing between recommending popular content and exploring less-known titles that might interest the user.
- Contextual Bandits: Incorporate context (e.g., time of day, user device) to make more informed recommendations.

Learn more about Netflix's research in this Netflix Research article on Recommendations.

Personalization Features on Netflix

Netflix's personalization extends beyond just what content to recommend: it also shapes how the content is presented. From homepage layout to visuals and timings, every touch point is thoughtfully tailored to enhance the viewers' experience:

- **Homepage Rows**: No two users see the same homepage. Netflix dynamically curates rows like "Because You Watched," "Top Picks," or "Continue Watching," with both the content and the row order adapting in real time to reflect users' interests and recent watches.
- **Artwork Personalization**: The personalization doesn't just stop at the homepage layout; even the thumbnail images are customized. For the same show or movie, different users may see completely different artwork- one highlighting romance, another emphasizing action or humor, depending on what themes have resonated most in their past viewing.
- **Contextual Recommendation**: Lastly, Netflix also considers the when and how of your watch list. For instance, if you're watching late at night, then Netflix may recommend shorter content or some familiar shows. But say, you opened Netflix on weekends, then expect deeper recommendations that have some sorta long-form storytelling.

Technical Infrastructure and Architecture

Netflix's recommendation engine is powered by a robust, scalable architecture:

Technical Architecture

1. Data Pipeline

- **Data Ingestion:** Massive amounts of data are ingested from various sources (user interactions, content metadata) in real-time.
- **Data Storage:** Data is stored in distributed systems like **Apache Cassandra** and **Amazon S3** to handle scale and ensure durability.
- **Data Processing: Apache Kafka** is used for real-time data streaming, while **Apache Spark** and **Flink** handle batch and stream processing.

2. **Model Training and Deployment**

- **Feature Engineering**: Complex features are engineered from raw data, such as user embeddings, content embeddings, and temporal features.
- **Model Training**: Models are trained on distributed computing frameworks like **TensorFlow**, **PyTorch,** and **Horovod**, leveraging GPU clusters.
- **Model Deployment**: Trained models are deployed using containerized environments (Docker, Kubernetes) to ensure scalability and reliability.
- **Continuous Learning**: The system continuously learns from new data. **Online learning algorithms and incremental training approaches** are used to keep models updated.

Compiled for EEE Final Year by Manikandan R
Compiled by: Manikandan R

3. **Scalability and Performance**

- Netflix employs a microservices architecture to handle different aspects of the recommendation system. This ensures modularity and ease of maintenance.
- Load balancing and caching strategies are implemented to handle high traffic volumes and ensure low latency in serving recommendations.

Evaluation and Testing

Netflix employs extensive **A/B testing** to evaluate the effectiveness of its recommendation algorithms. Different versions of the recommendation system are tested with various user segments to determine which version improves user engagement and satisfaction the most. Key metrics include:

- **Click-Through Rate (CTR)**: Measures how often users click on recommended content.
- **View Duration**: Tracks how long users watch the recommended content.
- **User Retention**: Analyzes the impact of recommendations on subscription renewals.

Impact on User Experience and Business

1. Increased User Engagement

Personalized recommendations keep users watching longer, reducing churn and increasing subscription longevity. Over 80% of viewing comes from personalized recommendations. Users spend less time searching and more time watching, improving retention.

2. Content Discovery

The recommendation system helps surface lesser-known titles, ensuring that even niche content finds its audience. This helps in maximizing the return on investment for all content acquired or produced.

3. Operational Efficiency

By accurately predicting user preferences, Netflix can make better-informed decisions about which new content to acquire or produce, optimizing content spend.

4. Customer Satisfaction

Personalized experiences lead to higher customer satisfaction, reflected in positive reviews and word-of-mouth referrals.

Challenges and Future Directions

- **Data Privacy**: Collecting and using personal data responsibly is crucial. Netflix ensures user data is anonymized and used ethically.
- **Diverse Preferences**: Balancing the diverse tastes of a global audience remains a challenge. Continuous improvements in localization and personalized algorithms are needed.
- **Algorithmic Bias**: Ensuring that recommendation algorithms do not reinforce existing biases is essential. Netflix works to maintain fairness and diversity in recommendations.

Compiled for EEE Final Year by Manikandan R
Compiled - Manikandan R

- **Real-Time Personalization**: Enhancing real-time personalization to provide instant recommendations based on immediate user actions and changing contexts.

Conclusion

Netflix's recommendation system is one of the most advanced in the world. By combining behavioral data, content metadata, and AI techniques, it delivers a hyper-personalized experience that keeps users engaged and satisfied.

From CNNs analyzing thumbnails to contextual bandits suggesting what to watch next, the depth of Netflix's personalization engine reflects its core mission: to connect members with stories they'll love faster and smarter. As AI and machine learning continue to evolve, Netflix will remain at the forefront of innovation in personalized content delivery. Because at Netflix, personalization isn't a feature- it's a complete product. And every scroll, click, and watch only makes it smarter.

Programming Tips for Large Datasets

Dealing with large datasets in data science requires strategies that move beyond in-memory processing. Here are essential programming tips and best practices for efficiency and scalability.

Programming Tips for Large Datasets

1. Optimize Data Structures and Types

The most fundamental step is to reduce the memory footprint of your data before processing.

- **Downcast Numeric Types:** By default, libraries like Pandas use 64-bit integers and floats. If your data fits, explicitly use smaller types to save significant memory:
  - Floats: Use `float32` instead of `float64`.
  - Integers: Use `int8`, `int16`, or `int32` instead of `int64`.
- **Use Categoricals for Low-Cardinality Strings:** Convert repetitive string columns (e.g., country codes, genders) to the Pandas `category` dtype. This stores strings as efficient integer codes, dramatically reducing memory usage.
- **Handle Missing Values Efficiently:** If you can't use smaller integer types due to NaN values (which force conversion to float), consider using libraries that support nullable integer types (like Pandas' Int64Dtype or PyArrow).

2. Learn Out-of-Core Processing

When data is too large to fit in RAM, you must process it chunk by chunk, often referred to as "out-of-core" or "streaming" processing.

- **Read Data in Chunks:** Use the `chunksize` parameter when reading files (e.g., Pandas'.read_csv(..., chunksize=100000)). Process, aggregate, or train your model on each chunk, then discard it, keeping only the results.
- **Generator Functions:** Write custom Python **generator functions** to yield chunks of data instead of loading the entire dataset into memory at once.

Compiled for EEE Final Year by Manikandan R

- **Use HDF5 or Parquet:** Store data in file formats designed for efficient, column-wise access and compression.
    - **Parquet** is highly recommended as it's a columnar storage format, enabling you to read only the columns needed for a task (predicate pushdown), which is a huge performance gain for wide datasets.

3. Leverage Scalable Libraries and Frameworks

Don't try to reinvent the wheel with pure Python loops; use libraries optimized for parallel and distributed computing.

- **Dask:** A native Python library that extends NumPy, Pandas, and Scikit-learn to larger-than-memory datasets. It creates task graphs for parallel execution, letting you use familiar Pandas syntax for massive datasets.
    - *Tip:* Use Dask DataFrames instead of Pandas DataFrames for large-scale operations.
- **PySpark/Spark:** Essential for truly massive, distributed datasets across a cluster. Spark's DataFrame API and MLlib are the industry standard for large-scale ETL and machine learning.
    - *Tip:* Prefer the high-level DataFrame operations over RDDs (Resilient Distributed Datasets) as DataFrames allow Spark to perform query optimization.
- **Vaex:** A high-performance DataFrame library that uses memory-mapping and an out-of-core approach to handle up to a billion rows efficiently, often used for visualization and quick statistics on massive data.

4. Efficient Modeling and Feature Engineering

Your modeling approach must also be scalable.

- **Avoid Memory-Heavy Models:** Algorithms that require the entire dataset to reside in memory (e.g.,k-Nearest Neighbors) may not be feasible.
- **Choose Online/Incremental Learning:** For huge datasets, use models that support **partial fit** or **incremental learning** (e.g., Stochastic Gradient Descent classifiers/regressors, Vowpal Wabbit).These models train by reading data one batch at a time, without needing the whole dataset upfront.
    - *Example:* In Scikit-learn, use `model.partial_fit(X_chunk, y_chunk)`.
- **Feature Hashing/Hashing Trick:** When dealing with high-cardinality categorical features (e.g., text, URLs), use the **Hashing Trick** to convert them into a fixed-size numerical vector without creating a large dictionary, saving both memory and processing time.

5. Prioritize Column-Wise Operations

Vectorized operations are the fastest way to work with data in Python, but they become even more crucial for large datasets.

- **NumPy and Pandas:** Always try to express operations as NumPy array operations or Pandas methods, which are executed in C/C++ under the hood, making them significantly faster than standard Python loops.
- **Avoid apply with Custom Functions:** While convenient, Pandas'.apply() can be slow. If you must use a custom function, try to use Pandas' `apply()` with the numba library to compile the function for faster execution.

Compiled for EEE Final Year by Manikandan R
Manikandan R